Application Analytics and History Dashboard (US-25)

Purpose of AI Use

Requirements analysis, implementation planning, coding, unit testing, acceptance-test design, and validation for the application analytics and history dashboard.

AI-Suggested Content

The AI reviewed US-25 (#160), the Sprint 3 project guidelines, related application/status-tracking stories, and existing application history. It drafted a user-story description with acceptance criteria and implementation tasks. The existing My Applications page was retained as the brief history view, with a link to a new dedicated analytics page.

The new `/applications/analytics` page shows a monthly bar chart of applications submitted over the past 12 months, current application counts by status, and application history filterable by current status. Each history entry includes the job title, company, application date, and current status. Analytics and history are scoped to the signed-in job seeker. No database changes or status-change timeline were added.

The AI extracted analytics calculations into `src/lib/server/application-analytics.ts` and added unit tests in `src/lib/server/application-analytics.test.ts`. It also added acceptance-style tests in `tests/application-analytics.test.ts` that exercise the actual page loader with a recording database stub, covering dashboard data, status filtering, preservation of overall metrics when filtered, and denial of recruiter and guest access.

Validation

The five unit tests passed. The three analytics acceptance tests passed. `npm run check` reported no errors or warnings, and `npm run build` completed successfully for the dashboard implementation. `git diff --check` reported no whitespace errors.

Decision

Accepted / Modified. The separate dashboard page and 12-month chart were implemented with native SVG rather than adding a charting dependency. The user deferred writing acceptance tests until after the dashboard implementation, then requested two acceptance-test scenarios and corresponding issue descriptions. The tests use a stub database and exercise the real page loader; they are not browser-driven end-to-end tests.

Reflection

The existing application rows already stored submission dates and current statuses, so the dashboard could reuse existing data without a schema migration. Extracting analytics logic made date grouping, status counting, and filtering independently unit-testable. Acceptance tests then verified that the page loader connects the calculations to owner-scoped application data and enforces the job-seeker access rule. The test issue descriptions were provided for the user to add to GitHub; no issues were created.

Responsible Person

Michael Derocher

Prompt 1

I'm working on the analytics page user story for the project. The related repo story is #160, but it does not have a description yet. Based on the Sprint 3 project guidelines in `SOEN341_CareerConnect_Sprint3.pdf` and the other user stories in the repository, draft a description for the application analytics and history dashboard.

Response 1

Reviewed #160, related application and application-status stories, existing story formats, and Feature 2 in the Sprint 3 guidelines. Drafted a user story for a dedicated job-seeker dashboard, with criteria for application history, current-status breakdown, applications-over-time chart, per-user access restriction, and an empty state, plus implementation tasks and notes connecting US-15 and US-20.

The user clarified that they wanted the proposed description only and would add it to GitHub themselves. No GitHub issue edit was made.

Prompt 2

Explain what would be needed to implement this kind of feature.

Response 2

Inspected the existing `/applications` page, application schema, shared status labels, navigation, and test setup. Found that the app already stores application submission dates and statuses and already restricts the existing history route to job seekers. Recommended adding a dedicated analytics page, calculating status totals and monthly submissions on the server, drawing a lightweight chart without a new dependency, retaining an accessible history list and empty state, and testing date grouping, totals, and access control.

Clarified that only the current application status is stored; a status-change timeline would require additional persistence, but the user confirmed that it was not needed.

Prompt 3

Implement the analytics feature as a separate page linked from the existing application history. Include application date and current status, with status filtering if possible. Focus only on implementation; tests would come later.

Response 3

Added a “View analytics and history” link to `/applications` and created `/applications/analytics`. The route queries the signed-in job seeker's applications with job and company details, computes per-status counts and the trailing 12 monthly submission counts, and filters the history list by status. The dashboard renders the monthly chart as SVG and displays application date and current status for each history row. No schema changes or status-change timeline were introduced.

The loader enforces the job-seeker role, scopes the query to the current user, and passes the results to the analytics helper:

```
const me = locals.user;
if (!me || me.role !== 'job_seeker') error(403, 'Only job seekers can view application analytics

const applications = await db
    .select({
        id: application.id,
```

```
        jobPostingId: jobPosting.id,
        title: jobPosting.title,
        company: jobPosting.company,
        appliedAt: application.createdAt,
        status: application.status
    })
    .from(application)
    .innerJoin(jobPosting, eq(application.jobPostingId, jobPosting.id))
    .where(eq(application.userId, me.id))
    .orderBy(desc(application.createdAt));

return getApplicationAnalytics(applications, url.searchParams.get('status'));
```

The page renders the monthly activity as an accessible SVG bar chart:

```
<svg role="img" aria-label="Bar chart of applications submitted each month over the past 12 month
    {#each data.months as month, index (month.key)}
        {@const x = 36 + index * 54}
        {@const barHeight = month.count === 0 ? 0 : Math.max(3, (month.count / data.maxMonthlyApp
        <rect x={x + 12} y={165 - barHeight} width="30" height={barHeight} rx="6">
            <title>{month.label}: {month.count} applications</title>
        </rect>
    {/each}
</svg>
```

`npm run check` and `npm run build` passed. Tests were deferred as requested.

Prompt 4

Increase spacing on the “Apply filter” button because the text looks squished.

Response 4

Added horizontal padding to give the button label more room:

```
<button type="submit" class={primaryButton + ' px-5 sm:w-auto'}>Apply filter</button>
```

Prompt 5

Create unit tests for the feature.

Response 5

Extracted dashboard calculations into `src/lib/server/application-analytics.ts` and added five tests in `src/lib/server/application-analytics.test.ts` covering current-status totals (including zero-count statuses), the trailing 12-month UTC date-grouping window and its boundaries, status filtering, invalid/empty filters, and empty application data. The page loader now uses the shared helper.

The helper validates status filters, counts applications by current status, and groups submission dates by month:

```
const statusFilter = isApplicationStatus(rawStatus) ? rawStatus : '';
const statusCounts = Object.keys(APPLICATION_STATUSES)
    .filter(isApplicationStatus)
    .map((status) => ({
        status,
        label: APPLICATION_STATUSES[status],
        count: applications.filter((application) => application.status === status).length
    }));

for (const application of applications) {
    const key = monthKey(application.appliedAt);
    const count = monthCounts.get(key);
    if (count !== undefined) monthCounts.set(key, count + 1);
}
```

Unit tests include a check that a selected status filters the history but does not reduce overall statistics:

```
const result = getApplicationAnalytics(allApplications, 'interview', now);

assert.equal(result.applications.length, 1);
assert.equal(result.applications[0].status, 'interview');
assert.equal(result.totalApplications, 2);
assert.equal(result.statusCounts.find(({ status }) => status === 'applied')?.count, 1);
```

All five unit tests passed, and `npm run check` reported no errors or warnings.

Prompt 6

Implement acceptance tests, at least two, and provide descriptions for separate GitHub issues using #151 and #152 as examples. Start with one acceptance test before doing the other.

Response 6

Added the first acceptance test in `tests/application-analytics.test.ts`. It exercises the real dashboard page loader with a recording database stub and verifies that a job seeker receives their application history, status totals, and monthly chart counts, and that the query is scoped to the signed-in user. The test passed and `npm run check` was clean.

Representative assertions check the displayed totals and SQL filtering to the signed-in job seeker:

```
assert.equal(result.totalApplications, 3);
assert.deepEqual(
    result.statusCounts.map(({ status, count }) => [status, count]),
    [
        ['applied', 1],
        ['under_review', 0],
        ['interview', 1],
        ['offered', 0],
        ['rejected', 1]
    ]
);
assert.match(queries[0].sql, /"application"\."user_id" = \$/);
assert.deepEqual(queries[0].params, ['job-seeker-1']);
```

Prompt 7

Add one more acceptance test.

Response 7

Added a test for filtering history by current status while keeping all-application totals and chart metrics unchanged. Also added a distinct authorization test confirming that recruiters and guests are denied access before a database query is made. All three acceptance tests in the file passed, and `npm run check` reported no errors or warnings.

The filter scenario checks that only matching history rows appear while all applications remain included in the chart and status totals:

```
assert.deepEqual(result.applications.map((item) => item.id), ['application-interview']);
assert.equal(result.totalApplications, 2);
assert.equal(result.statusCounts.find((item) => item.status === 'applied')?.count, 1);
assert.equal(result.months.reduce((sum, month) => sum + month.count, 0), 2);
```

The access-control scenario checks that unauthorized roles receive HTTP 403 without querying the database:

```
await assert.rejects(
    async () => load({ locals: { user }, url: new URL('http://localhost/applications/analytics')
    (error) => isHttpError(error, 403)
);
assert.equal(queries.length, 0);
```